

Complements of Operational Research

5.2 Dynamic programming: the knapsack problem

José Firmino Aguilar Madeira

The knapsack problem

There is a limited capacity W and several types of items, each with weight w_i and value v_i . Which items should be taken to **maximise the value** without exceeding the capacity?

- **Unbounded knapsack:** any number of units of each type can be taken.
- **0–1 knapsack:** each item is either taken or not (one unit).

First idea: choose by the highest value per unit of weight, v_i/w_i . It works sometimes — but not always.

Where it appears

- loading lorries, containers, aircraft;
- project selection with a limited budget;
- cutting material (coils, sheets);
- choosing adverts for a commercial break;
- memory and bandwidth allocation.

It is an integer optimisation problem; DP solves it with a table.

Example 1: loading a lorry (unbounded knapsack)

A lorry carries up to 10 t. There are three types of pallets, in sufficient numbers:

Pallet	w_i (t)	v_i (hundreds €)	v_i/w_i
A	3	7	2.33
B	4	13	3.25
C	6	17	2.83

By ratio: B, B (8 t, profit 26); 2 t are left, where nothing fits.

Recursion

$f(k)$ = maximum profit with capacity k :

$$f(k) = \max_{i: w_i \leq k} \{v_i + f(k - w_i)\},$$

with $f(k) = 0$ if nothing fits.

State = free capacity k ; decision = which pallet to load next. Compute $f(0), f(1), \dots, f(10)$ in this order: each value uses values already computed.

The unbounded knapsack table

k	A: $7 + f(k - 3)$	B: $13 + f(k - 4)$	C: $17 + f(k - 6)$	$f(k)$	best
0-2	—	—	—	0	—
3	7	—	—	7	A
4	7	13	—	13	B
5	7	13	—	13	B
6	14	13	17	17	C
7	20	20	17	20	A or B
8	20	26	17	26	B
9	24	26	24	26	B
10	27	30	30	30	B or C

Reading the solution from $k = 10$: $f(10) = 13 + f(6)$
→ load B, 6 t left; $f(6) = 17 + f(0)$ → load C, 0 left.
(Column C gives the same: C and then B.)

Result

One pallet B and one C: 10 t, profit 30 —
against 26 by ratio. By brute force, only this load
reaches 30.

Example 2: choosing projects within a budget (0–1 knapsack)

A city council has 10 (hundreds of thousands of €) to invest and five candidate projects; each one is either carried out or not.

Project	Cost w_i	Benefit v_i	v_i/w_i
P1 cycle lane	3	12	4.0
P2 swimming pool	5	14	2.8
P3 LED lighting	3	15	5.0
P4 solar panels	4	15	3.75
P5 digitalisation	1	8	8.0

Benefit: estimated net present value.

By ratio: P5, P3, P1 (cost 7, benefit 35); P4 and P2 no longer fit.

Formulation in stages

Stage i = project i ; state k = budget still available; decision $x_i \in \{0, 1\}$.

$$f_i(k) = \max \left\{ \underbrace{f_{i+1}(k)}_{\text{no}}, \underbrace{v_i + f_{i+1}(k - w_i)}_{\text{yes, if } w_i \leq k} \right\}$$

with $f_6(k) = 0$. Solved from P5 to P1.

The 0–1 knapsack table

budget k	0	1	2	3	4	5	6	7	8	9	10
f_5 (P5 only)	0	8	8	8	8	8	8	8	8	8	8
f_4 (P4, P5)	0	8	8	8	15	23	23	23	23	23	23
f_3 (P3–P5)	0	8	8	15	23	23	23	30	38	38	38
f_2 (P2–P5)	0	8	8	15	23	23	23	30	38	38	38
f_1 (all)	0	8	8	15	23	23	27	35	38	38	42

Reading the solution ($k = 10$):

- P1: no $f_2(10) = 38$, yes $12 + f_2(7) = 42 \rightarrow$ **yes**; 7 left.
- P2: no $f_3(7) = 30$, yes $14 + f_3(2) = 22 \rightarrow$ no.
- P3: no $f_4(7) = 23$, yes $15 + f_4(4) = 30 \rightarrow$ **yes**; 4 left.
- P4: no $f_5(4) = 8$, yes $15 + f_5(0) = 15 \rightarrow$ **yes**; 0 left.
- P5: does not fit.

Result

P1, P3 and P4: cost 10, benefit **42** — against 35 by ratio.

Note: the most “efficient” project (P5) is left out — it would use 1 of the budget needed for P4.

Unbounded or 0–1: what changes

	Unbounded	0–1
State	free capacity k	(item i , free capacity k)
Recursion	$f(k) = \max_i \{v_i + f(k - w_i)\}$	$f_i(k) = \max\{f_{i+1}(k), v_i + f_{i+1}(k - w_i)\}$
Table	one row ($W + 1$ values)	one row per item ($n \times (W + 1)$)
Effort	about nW additions	about nW additions

- In the 0–1 case we must remember **which items have already been used**: that is why the state includes the stage i .
- The effort grows with W : with very large capacities (or weights with decimal places) the table becomes huge. Then *branch and bound* (ch. 3) or integer linear programming solvers are used.

In Python, and confirmation by integer linear programming

```
def mochila_01(w, v, W):
    n = len(w)
    F = [[0] * (W + 1) for _ in range(n + 1)]
    for i in range(n - 1, -1, -1):          # P5 -> P1
        for k in range(W + 1):
            F[i][k] = F[i + 1][k]          # no
            if w[i] <= k:                  # yes
                F[i][k] = max(F[i][k],
                               v[i] + F[i + 1][k - w[i]])

    x, k = [], W
    for i in range(n):
        x.append(int(F[i][k] != F[i + 1][k]))
        k -= w[i] * x[-1]
    return F[0][W], x

print(mochila_01([3, 5, 3, 4, 1], [12, 14, 15, 15, 8], 10))
# (42, [1, 0, 1, 1, 0])
```

```
from scipy.optimize import milp, LinearConstraint
import numpy as np
w = np.array([3, 5, 3, 4, 1])
v = np.array([12, 14, 15, 15, 8])
r = milp(-v,
          constraints=LinearConstraint(w, 0, 10),
          integrality=np.ones(5), bounds=(0, 1))
print(-r.fun, r.x)      # 42.0 [1. 0. 1. 1. 0.]
```

The same answer by both methods. For the unbounded knapsack just replace bounds=(0, 1) with (0, np.inf).

Example 4 — the ship

A ship with a capacity of 4 t can carry one or more of the products in the table (w_i in tonnes; R_i , unit revenue in thousands of €). Find the number of units of each product that maximises the revenue.

Product	w_i	R_i
1	2	31
2	3	47
3	1	14

Examples 5 and 6 — knapsack (several units of each type)

Ex. 5: 10 kg			Ex. 6: 13 kg		
Prod.	Weight	Value	Prod.	Weight	Value
A	3	6	A	3	6
B	4	7	B	4	7
C	7	10	C	5	10
			D	6	12

Which products, and how many of each, should be put in the knapsack to maximise the value?

KEY TAKEAWAYS — 5.2

1. Unbounded knapsack: $f(k) = \max_i \{v_i + f(k - w_i)\}$, for $k = 0, \dots, W$.
2. 0–1 knapsack: stage = item, state = free capacity;
 $f_i(k) = \max\{f_{i+1}(k), v_i + f_{i+1}(k - w_i)\}$.
3. The solution is read backwards in the table, starting from $f(W)$ (or $f_1(W)$).
4. Choosing by the ratio v_i/w_i can fail (26 against 30; 35 against 42).
5. Confirm by brute force (few items) or by integer linear programming.

Next: 5.3 Workforce planning

References Full list: *Course bibliography*.

- Hillier, F. S., Lieberman, G. J., *Introduction to Operations Research*, 11th ed., McGraw-Hill, 2021 — Dynamic Programming.
- Kellerer, H., Pferschy, U., Pisinger, D., *Knapsack Problems*, Springer, 2004.
- Taha, H. A., *Operations Research: An Introduction*, 10th ed., Pearson, 2017 — the cargo-loading model (*knapsack*).

Resources

- Exercise sheet 5. `code/python/05_dynamic_programming/05_2_knapsack/mochila.py`.